

Week_4

Task 1: Write a C program where a parent process creates multiple child processes and synchronizes their completion using `wait()` and `waitpid()`. Compare the behavior of both functions.

ALGORITHM: Parent Creates Multiple Children and Synchronizes using waitpid()	
1.	START
2.	Declare an array <code>child[NUM_CHILDREN]</code> to store child PIDs and an integer status.
3.	Print a message indicating the parent process has started along with its PID.
4.	For $i \leftarrow 0$ to <code>NUM_CHILDREN - 1</code> do 4.1 Call <code>fork()</code> and store the return value in <code>child[i]</code>. 4.2 If <code>fork() < 0</code> then 4.3 Print error message and terminate the program. 4.4 Else if <code>fork() == 0</code> then // Child process 4.4.1 Print message: "Child $i + 1$ started" along with child PID. 4.4.2 Sleep for $(i + 1) \times 2$ seconds (to create different completion times). 4.4.3 Print message: "Child $i + 1$ completed". 4.4.4 Exit the child process with exit status $(i + 1)$.
5.	End For
6.	Print message: Parent waiting for children using <code>waitpid()</code> .
7.	For $i \leftarrow 0$ to <code>NUM_CHILDREN - 1</code> do
8.	Call <code>waitpid(child[i], &status, 0)</code> and store the return value in <code>pid</code> .
9.	If <code>waitpid() == -1</code> then
10.	Print error message and terminate the program.
11.	Else
12.	If <code>WIFEXITED(status)</code> is true then
13.	Print: Child with PID <code>pid</code> terminated with exit status <code>WEXITSTATUS(status)</code> .
14.	End If
15.	End If
16.	End For
17.	Print message: All child processes completed. Parent exiting. STOP

Sample Output:

Parent Process Started

Parent PID: 4521

Child 1 started. PID = 4522, Parent PID = 4521

Child 2 started. PID = 4523, Parent PID = 4521

Child 3 started. PID = 4524, Parent PID = 4521

All 3 child processes have been created.

Using `wait()`:

Child 1 completed. PID = 4522

wait() detected child PID 4522 finished with exit status 10

Using waitpid():

Child 2 completed. PID = 4523

Child 3 completed. PID = 4524

waitpid() detected child PID 4522 finished with exit status 10

waitpid() detected child PID 4523 finished with exit status 11

waitpid() detected child PID 4524 finished with exit status 12

Parent process completed.

Aim

To create multiple child processes using fork() and synchronize their completion using wait() and waitpid().

PROGRAM

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
#include <sys/wait.h>
```

```
int main() {
```

```
    pid_t children[3];
```

```
    int status;
```

```
    printf("Parent Process Started\n");
```

```
    printf("Parent PID: %d\n", getpid());
```

```
    // Create 3 child processes
```

```
    for (int i = 0; i < 3; i++) {
```

```
        children[i] = fork();
```

```
        if (children[i] < 0) {
```

```
            perror("fork failed");
```

```
            exit(1);
```

```
        }
```

```

    if (children[i] == 0) {
        printf("Child %d started. PID = %d, Parent PID = %d\n",
            i + 1, getpid(), getppid());

        sleep(i + 1);

        printf("Child %d completed. PID = %d\n", i + 1, getpid());

        exit(10 + i);
    }
}

printf("All 3 child processes have been created.\n\n");

// Using wait()
printf("Using wait():\n");

pid_t pid = wait(&status);

if (WIFEXITED(status)) {
    printf("wait() detected child PID %d finished with exit status %d\n",
        pid, WEXITSTATUS(status));
}

// Using waitpid()
printf("\nUsing waitpid():\n");

for (int i = 1; i < 3; i++) {
    pid = waitpid(children[i], &status, 0);

    if (WIFEXITED(status)) {
        printf("waitpid() detected child PID %d finished with exit status %d\n",
            pid, WEXITSTATUS(status));
    }
}

printf("Parent process completed.\n");

```

```
    return 0;
}
```

Output

```
amrutha@blue:~$ cd ~/OSSP/Week3
amrutha@blue:~/OSSP/Week3$ mkdir ~/OSSP/Week4
mkdir: /home/amrutha/OSSP/Week4: File exists
amrutha@blue:~/OSSP/Week3$ cd ~/OSSP/Week4
amrutha@blue:~/OSSP/Week4$ pwd
/home/amrutha/OSSP/Week4
amrutha@blue:~/OSSP/Week4$ nano wait_waitpid.c
amrutha@blue:~/OSSP/Week4$ gcc wait_waitpid.c -o wait_waitpid
amrutha@blue:~/OSSP/Week4$ ./wait_waitpid
Parent Process Started
Parent PID: 632
Child 1 started. PID = 633, Parent PID = 632
Child 2 started. PID = 634, Parent PID = 632
All 3 child processes have been created.

Child 3 started. PID = 635, Parent PID = 632
Using wait():
Child 1 completed. PID = 633
wait() detected child PID 633 finished with exit status 10

Using waitpid():
Child 2 completed. PID = 634
waitpid() detected child PID 634 finished with exit status 11
Child 3 completed. PID = 635
waitpid() detected child PID 635 finished with exit status 12
Parent process completed.
amrutha@blue:~/OSSP/Week4$ |
```

Observation

The parent process successfully created three child processes. `wait()` was used to wait for a child process to complete, while `waitpid()` was used to wait for specific child processes.

Result

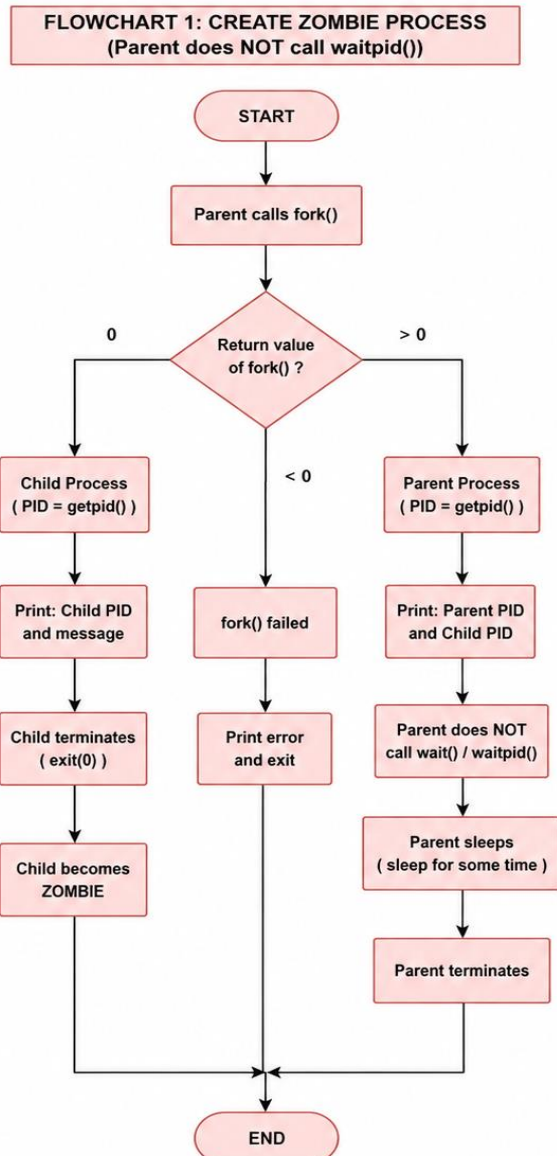
The parent and child processes were successfully synchronized using `wait()` and `waitpid()`.

Task 2: Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

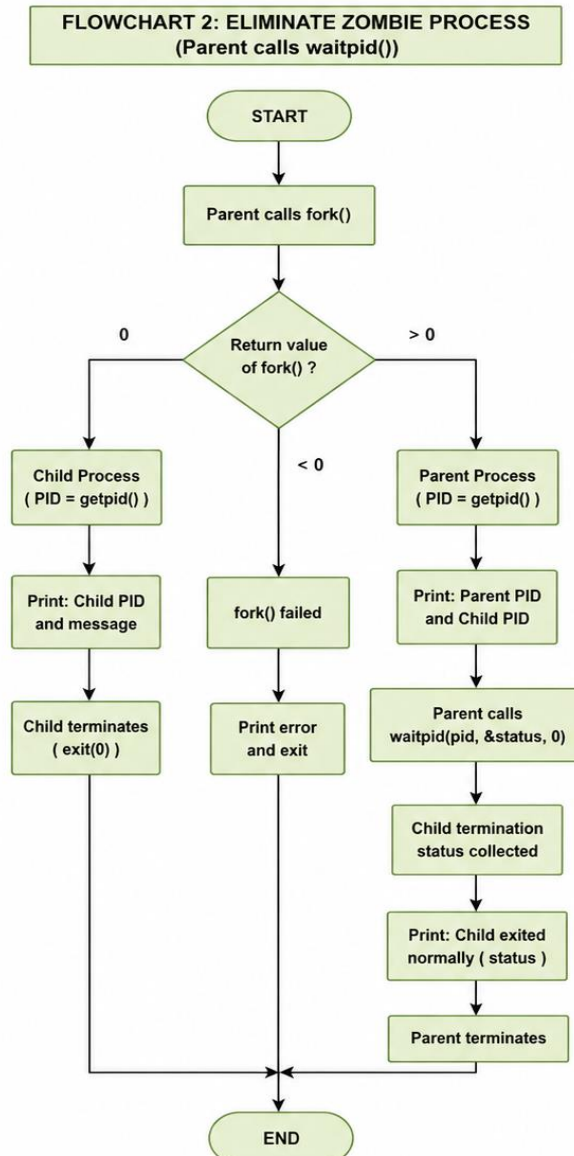
Aim

To create and observe a zombie process and eliminate it using proper process synchronization.

Flow Chart:



During the time parent is sleeping,
child is shown in process table as Z (zombie / <defunct>)



Since parent calls waitpid(),
child does NOT remain a zombie.

Sample Output:

Parent process: PID = 158

Child process: PID = 159

Child PID = 159

Child process terminating...

Parent sleeping...

Parent terminating...

Ps -el

F	S	UID	PID	PPID	C	PRI	NI	ADDR	SZ	WCHAN	TTY	TIME	CMD
0	S	0	1	0	0	80	0	- 2238 ?	?			00:00:00	init
0	S	0	8	1	0	80	0	- 2238 ?		tty1		00:00:00	init
0	S	1000	9	8	0	80	0	- 3590 ?		tty1		00:00:00	bash
0	S	0	141	1	0	80	0	- 2238 ?		tty2		00:00:00	init
0	S	1000	142	141	0	80	0	- 3527 ?		tty2		00:00:00	bash
0	S	1000	158	9	0	80	0	- 2680 ?		tty1		00:00:00	week4_T2
0	Z	1000	159	158	0	80	0	- 0 ?		tty1		00:00:00	week4_T2
0	R	1000	160	142	0	80	0	- 4086 ?		tty2		00:00:00	ps

PROGRAM

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
int main() {  
    pid_t pid = fork();  
  
    if (pid < 0) {  
        perror("fork failed");  
        return 1;  
    }  
  
    if (pid == 0) {  
        printf("Child process: PID = %d\n", getpid());  
        printf("Child process terminating...\n");  
        exit(0);  
    }  
    else {  
        printf("Parent process: PID = %d\n", getpid());  
        printf("Child PID = %d\n", pid);  
        printf("Parent sleeping...\n");  
  
        sleep(20);  
  
        printf("Parent terminating...\n");  
    }  
  
    return 0;  
}
```

Output

```
amrutha@blue:~/OSSP/Week4$ nano zombie.c
amrutha@blue:~/OSSP/Week4$ gcc zombie.c -o zombie
amrutha@blue:~/OSSP/Week4$ ./zombie
Parent process: PID = 919
Child PID = 920
Parent sleeping...
Child process: PID = 920
Child process terminating...
Parent terminating...
amrutha@blue:~/OSSP/Week4$ |
```

```
amrutha@blue:~$ ps -o pid,ppid,state,stat,cmd -p 920
PID    PPID  S  STAT  CMD
920    919  Z  Z+    [zombie] <defunct>
```

Observation

The child process terminated while the parent process was still running. During this time, the child appeared in the process table with the Z state, indicating a zombie process.

Result

A zombie process was successfully created and observed. Proper synchronization using `wait()` can be used to prevent the child from remaining as a zombie.